Simulação de um Sistema Operacional Multitarefa de Tempo Compartilhado

Versão 0.7 – 15/10/26

Autor

Prof. Dr. Marco Aurélio Wehrmeister

Objetivos

1. Aprofundar o estudo do gerenciamento de tarefas através de implementação de escalonadores, a execução de múltiplas tarefas e do escalador, e a apresentação gráfica dessa execução do sistema ao longo do tempo.
2. Aprimorar o conhecimento sobre interação entre tarefas usando serviços de exclusão mútua comumente encontrados em sistemas operacionais multi-tarefa;
3. Compreender e resolver os problemas de escalonamento, p.ex., inanição, inversão de prioridades, impasse, etc.

Requisitos Gerais

Todos os requisitos abaixo devem ser obrigatoriamente contemplados em AMBOS os projetos:

1. O projeto deve implementar um **software que simula a execução** das tarefas em um ***sistema operacional multitarefa preemptivo de tempo compartilhado***;
2. O computador possui **mais de um processador capaz de executar as tarefas** do sistema. A quantidade de processadores é configurada pelo usuário, sendo, no mínimo 01 (um) e no máximo N processadores.
3. O software deve permitir a visualização gráfica da execução das tarefas dentro do sistema operacional ao longo do tempo para cada processador. **Todos os eventos importantes/relevantes** que acontecem ao longo da execução das tarefas devem **aparecer claramente na visualização gráfica** na forma de um elemento gráfico distinto para cada tipo de evento. Uma legenda destes elementos gráficos deve ser mostrada na visualização. A visualização deve indicar, claramente, em qual processador cada tarefa está executando.

4. O software deve ser configurável e permitir que o usuário controle a simulação do sistema, bem como as características do sistema que está sendo simulado. Deve ser possível avançar e retroceder a simulação, sendo que o estado atual de cada tarefa em execução pode ser modificado manualmente em qualquer passo/estágio da simulação.
5. **A linguagem a ser utilizada para implementação do projeto é de livre escolha, no entanto, é necessário que o software criado possa ser executado sem a necessidade de instalar nenhuma biblioteca extra, ou qualquer outro software/runtime, ou equivalente.**
6. Todo o código fonte deve ser comentado de forma que qualquer outra pessoa consiga compreender o que cada parte/função/componente de código faz e as razões/justificativas de todas as decisões/escolhas de implementação;

PROJETO A – Implementação Inicial do Simulador

Requisitos:

1. O projeto deve implementar um software que simula a execução de tarefas em um sistema operacional multitarefa preemptivo de tempo compartilhado.
    1.1. O sistema simulado deve **incluir um relógio global do sistema**, sendo que a passagem do tempo deve ser simulada na forma de *ticks* de relógio. A simulação deve verificar se existe a necessidade de realizar alguma ação ou evento (ou algo relevante para a simulação) a cada *tick* do relógio. A passagem do tempo na simulação não precisa estar atrelada ao tempo no mundo real, ou seja, o tempo de simulação pode ser incrementado/decrementado sem esperar a mesma quantidade de tempo passar no mundo real.
    1.2. O sistema possui **múltiplos processadores/CPUs/cores para executar as tarefas**. O escalonador deve **minimizar a ociosidade destes processadores**, ou seja, não devem existir tarefas na fila de prontos e CPUs sem uso. A fila de prontos é global do sistema, ou seja, não é individual para cada CPU. Caso não seja possível atribuir uma tarefa para executar em um processador, o processador deve ser desligado. A informação sobre o período de tempo que um processador está desligamento deve ser apresentada para o usuário e também na visualização gráfica da execução das tarefas.
    1.3. As informações de cada tarefa (antes, durante, depois da simulação) devem ser armazenadas durante a simulação em uma única estrutura de dados, p.ex., *Task Control Block* (TCB);
    1.4. A simulação da execução das tarefas deve seguir as características de tempo informadas para cada tarefa, p.ex., instante de ingresso, duração, instante da ocorrência de eventos, etc.
    1.5. **Dois modos de execução** da simulação devem ser implementados: (a) execução **passo-a-passo**; (b) execução **completa** sem intervenção humana.
        1.5.1. No caso execução passo-a-passo (opção a), deve ser possível examinar o estado atual e as informações correntes do sistema e também de cada tarefa individualmente. A ideia é que esse modo de execução forneça um *debugger* do sistema.
        1.5.2. No caso execução passo-a-passo (opção a), o simulador deve permitir que o usuário avance e retroceda a simulação conforme seu desejo. Dessa forma, o histórico do estado do sistema, a cada passo da execução da simulação, deve ser armazenado na memória e deve ser usado no processo de

retroceder/avançar a simulação. **Retroceder/avançar a simulação tem impacto no relógio global do sistema**. Além disso, simulador deve **permitir que o usuário <u>modifique</u> o estado atual ou as características de <u>qualquer tarefa</u>** do sistema em qualquer passo/estágio da simulação.

1.5.3. No caso execução completa (opção b), o software mostra apenas o resultado final da simulação do sistema. Não é necessário mostrar os passos intermediários da simulação.

2. O software deve permitir a visualização gráfica da execução das tarefas dentro do sistema operacional ao longo do tempo.

   2.1. Apresentar um gráfico de *gantt* que mostra as tarefas sendo executadas ao longo do tempo e o processador usado em cada período de execução. Quando uma tarefa está executando, tal informação deve ser indicada com uma cor (definida em um parâmetro de entrada) e cada tarefa deve ter uma cor diferente. Quando uma tarefa está na fila de prontas e ela não está sendo executada, tal informação deve ser indicada como a ausência da cor. Quando uma tarefa está suspensa, por qualquer motivo, tal informação deve ser indicada com a cor preta e preenchimento diferente daquele usado para indicar a execução da tarefa (p.ex. preenchimento com pontos ao invés de sólido).

   2.2. Quando uma tarefa nova chega no sistema isso deve ser claramente mostrado no gráfico de *gantt*. Da mesma forma, quando a tarefa termina a sua execução, isso deve ser claramente mostrado no gráfico de *gantt*. Para isso utilize ícones, figuras, setas ou outro elemento gráfico que indique esses eventos. **Esses eventos devem ser mostrados no momento exato em que eles acontecem.**

   2.3. A atualização do gráfico de *gantt* deve ser compatível com o modo de execução da simulação (conforme requisito 1.4). Na execução passo a passo, o gráfico de gantt atualizado deve ser mostrado na tela para o usuário a cada passo da simulação.

   2.4. Ao final da simulação, o software deve gerar um arquivo do tipo imagem (p.ex. JPG, PNG, SVG, etc) contendo o gráfico de gantt correspondendo a situação final do sistema após a execução de todas as tarefas. **A figura gerada <u>não pode ser um print</u> da janela que mostra a simulação, deve conter a imagem que representa toda a simulação e não deve estar limitada a nenhum tempo máximo de simulação.**

   2.5. A visualização do gráfico de *gantt* deve ser similar as figuras da seção 6.4 do livro do Prof. Maziero. A ordem em que as tarefas aparecem no eixo Y deve ser decrescente em relação ao ID, ou seja, a tarefa com ID menor é a mais próximas do eixo X, a tarefa com o segundo ID menor aparece em cima da primeira, e assim por diante.

3. O software deve ser configurável e permitir que o usuário controle a simulação do sistema, bem como as características do sistema que está sendo simulado.
    3.1. O software deve permitir a configuração/parametrização do sistema a ser simulado antes e durante a execução da simulação, incluindo, mas não limitado a: conjunto de tarefas, características individuais de cada tarefa, algoritmo de escalonamento, eventos como solicitação/liberação de um mutex, operação de entrada/saída, envio/recebimento de dados, entre outros. Caso alguma modificação não faça sentido ou não possa ser aplicada durante a execução, o usuário precisa ser informado claramente que a modificação não pode ser feita, justificando o motivo.
    3.2. O software deve sugerir valores padrão para todo os parâmetros de configuração da simulação ou do software em si. Esses valores padrão devem ser sobrescritos por valores indicados pelo usuário, tanto na hora de carregar o arquivo de configuração como também nas telas que permite a modificação dos parâmetros da simulação.
    3.3. A configuração/parametrização do sistema a ser simulado deve ser carregada a partir de um arquivo texto, formato texto simples (plain text), e deve seguir o seguinte formato:

```
algoritmo_escalonamento;quantum;qtde_cpus
id;cor;ingresso;duracao;periodo;prazo;lista_eventos
```

onde:

`algoritmo_escalonamento` indica o algoritmo de escalonamento a ser usado na simulação;

`quantum` indica o período máximo de tempo que uma tarefa pode executar;

`qtde_cpus` é a quantidade de processadores que o sistema possui para executar as tarefas (valores entre 1 e N);

`id` é o identificador único da tarefa pode executar;

`cor` é cor que identifica a execução da tarefa no formato RGB em Hexadecimal, p.ex. “F0E0D0” indica Red=F0, Green=E0, Blue=D0

`ingresso` indica o instante de tempo que a tarefa foi criada;

`duracao` indica o tempo de execução da tarefa;

`periodo` indica o período no qual uma tarefa é ativada: valores maiores que zero indicam que a tarefa tem ativação periódica a cada “n” *ticks* de relógio; valor zero indica que a tarefa é aperiódica, ou seja, é ativada apenas uma vez no momento da sua criação; valores menores que zero representam erros no arquivo.

`prazo` indica o tempo máximo que uma ocorrência/execução da tarefa deve terminar a sua execução (*deadline*). O prazo começa a contar a partir do momento que a tarefa for ativada. Caso o tempo de execução da ocorrência/execução da tarefa ultrapasse o prazo (deadline), isso indica um erro e deve ser mostrado no gráfico de *gantt*. A tarefa que perdeu o prazo continua sendo executada e só termina quando o tempo de execução da tarefa for igual ou maior que a duração da tarefa.

`lista_eventos` indica uma lista de eventos que ocorre durante a execução da tarefa.

3.3.1. A primeira linha do arquivo de configuração indica parâmetros gerais da simulação ou do sistema operacional. A partir da segunda linha, cada linha indica os parâmetros de uma tarefa do sistema. O arquivo de configuração deve ter pelo menos duas linhas (primeira linha: parâmetros do sistema; a partir da segunda linha: parâmetros de uma tarefa, uma por linha), sendo que não há limite máximo para o número de tarefas a serem executadas no sistema.

3.3.2. Todas as *strings* contidas no arquivo de configurações devem ser tratas de forma igual para o caso possuírem letras maiúsculas, minúsculas, ou a mistura de ambos. Exemplo: “PRIOP”, “priop” ou “PrioP” deve indicar a mesma configuração: escalonador preemptivo por prioridades.

3.3.3. Qualquer linha do arquivo de configuração pode terminar com o caracter “;” ou não. Os dois casos devem ser tratados no simulador.

3.3.4. O simulador deve conseguir carregar um arquivo de configuração com qualquer nome, a partir de qualquer lugar no sistema de arquivos do computador, incluindo, mas não limitado a: qualquer diretório da partição principal ou qualquer outra partição do disco do computador ou de um disco externo (ex. pendrive). Não é permitido utilizar um arquivo fixo como arquivo de configuração.

3.3.5. A lista de eventos de cada tarefa vai ser tratada no projeto B. No entanto, deve-se atentar para o fato da possibilidade de ocorrência de inúmeros eventos ao longo da execução de uma tarefa.

3.3.6. Caso existam erros de carregamento do arquivo de configuração, ou erros no próprio arquivo de configuração, eles devem ser claramente mostrados ao usuário indicado claramente o motivo do problema. Espaços e linhas em branco não representam erros e devem ser ignorados.

3.3.7. Qualquer erro ao carregar o arquivo de configuração válido (ou seja, que obedeça ao formato estabelecido nos requisitos) ou que impossibilitem

carregar qualquer arquivo de configuração válido indicam falha grave no projeto e impedirão a realização da sua demonstração.

3.4. O simulador deve permitir que o usuário modifique o estado atual de qualquer tarefa do sistema em qualquer passo/estágio da simulação. Caso a modificação solicitada seja semanticamente inválida ou não seja possível de ser aplicada, isso deve ser indicado claramente para o usuário.

4. Os seguintes algoritmos de escalonamento devem ser implementados: *Rate Monotonic* preemptivo (*string* "RM" no arquivo de configurações), *Earliest Deadline First* preemptivo (*string* "EDF" no arquivo de configurações) - ver capítulo "Escalonamento de Tempo-Real" no Moodle.

   4.1. O usuário deve indicar, antes da simulação, qual é o algoritmo de escalonamento a ser usado;

   4.2. O mecanismo de implementação do algoritmo de escalonamento deve ser flexível/configurável/parametrizável de modo que novos algoritmos possam ser facilmente incluídos no simulador sem a necessidade de modificar o código da simulação. Idealmente, o escalonador é apenas uma função que retorna qual é a próxima tarefa a ser executada, que pode, inclusive, estar contida em uma biblioteca dinâmica fora do arquivo binário do executável do simulador;

   4.3. Em caso de empates na escolha da próxima tarefa (para todos os algoritmos), o desempate deve considerar os seguintes critérios, nessa ordem: (1) a tarefa selecionada é a que está executando imediatamente antes do escalonamento ser acionado (para evitar uma troca de contexto para a própria tarefa); (2) o prazo da tarefa; (3) o instante de ingresso da tarefa: quem chegou antes é escolhida; (4) duração da tarefa: duração menor deve se escolhida; (5) sorteio (quando isso acontecer, um elemento gráfico deve ser mostrado no gráfico de *gantt* no momento em que a tarefa escolhida (re)começar a execução).

   4.4. No projeto A, a simulação deve suportar somente a execução de tarefas periódicas, sendo que as tarefas aperiódicas serão abordadas no projeto B. Caso exista a indicação de tarefas aperiódicas no projeto A, elas devem ser ignoradas e essa situação deve ser reportada ao usuário de forma clara. Por definição, as tarefas periódicas nunca terminam. No entanto, com objetivo de terminar a simulação, assume-se que uma tarefa periódica termina após ela ter executado 10 vezes.

5. A interface de interação do usuário com o simulador deve ser intuitiva, fácil de entender e usar. Qualquer erro que ocorrer durante o uso do software do simulador deve ser apresentado de forma clara para o usuário indicado o motivo da mensagem de erro ser apresentada.

PROJETO B – Refinamento da Simulação do Sistema Operacional Multitarefa

Requisitos:

1. Ampliar os tipos de tarefas suportados na simulação, incluindo a criação/execução de tarefas aperiódicas e esporádicas (ver capítulo "Escalonamento de Tempo-Real").
    1.1. A tarefas periódicas ou esporádicas não devem interferir no escalonamento das tarefas periódicas, ou seja, a sua execução deve ser feita usando algum dos servidores apresentados no capítulo "Escalonamento de Tempo-Real" a escolha do usuário.
    1.2. No arquivo de configuração, uma tarefa aperiódica é indicada como tendo o valor do `periodo` igual 0. Este valor deve ser guardado na TCP da tarefa.
    1.3.No arquivo de configuração, uma tarefa esporádica é indicada como tendo um valor negativo no campo `periodo`. O valor absoluto do campo `periodo` indica o período mínimo entre duas ocorrências consecutivas da tarefa. Cada ocorrência da tarefa esporádica é informada no campo `lista_eventos` da seguinte forma: `instante01,instante02,...` onde cada instante é um tempo relativo ao instante de criação da tarefa, p.ex. para uma tarefa criada no tempo 10, o valor zero indica que ela já está pronta para executar e o valor 5 indica que ela só fica pronta para executar no tempo 15. Ao ler a linha de configuração, deve-se realizar uma verificação da propriedade de tempo mínimo entre duas execuções consecutivas da tarefa.

2. O usuário pode indicar um ou mais servidores como sendo tarefas periódicas no sistema. O tipo do servidor é indicado no ID da tarefa como: (a) BSxx indica o *Background Server*; (b) PSxx indica o *Pooling Server*; (c) DSxx indica o *Deferrable Server*; e (d) SSxx indica o *Sporadic Server*; "xx" indica um numero inteiro para diferenciar os servidores de um mesmo tipo.
    2.1. Os demais campos da linha de configuração permanecem com a mesma semântica e definem como cada servidor vai se comportar (ver seção 2.8 do capítulo "Escalonamento de Tempo-Real").
    2.2. Todos os servidores tem acesso a fila de tarefas aperiódicas ou esporádicas prontas para serem executadas e pode escolher uma delas para executar (conforme a sua política de escalonamento), retirando-a da fila e tornando-a inacessível para outros servidores.

3. Executar o teste de escalabilidade do sistema antes de qualquer tarefa ser aceita e incluída para execução no sistema, incluindo a avaliação todos os tipos de tarefas que são criadas durante a execução do sistema, p.ex., tarefas periódicas, aperiódicas, esporádicas e servidores.

4. A execução das tarefas aperiódicas/esporádicas e dos servidores deve ser apresentada de forma que se possa visualizar claramente no gráfico de gantt quando o servidor está executando e quando as tarefas aperiódicas/esporádicas estão executando. Sugestão: usar um padrão de preenchimento diferente de sólido, ou usar uma borda diferente em espessura ou padrão de desenho.